

Section 1. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the B
Section 2. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman
Section 3. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditin
Section 4. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Co
Section 5. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybr
Section 6. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding

Section 7. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-le
Section 8. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three
Section 9. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural bl
Section 10. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 11. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns throu
Section 12. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when th

Section 13. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with
Section 14. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns
Section 15. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting e
Section 16. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, a
Section 17. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tie
Section 18. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual cou

Section 19. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met.

Section 20. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met.

Section 21. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met.

Section 22. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met.

Section 23. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met.

Section 24. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision is met.

Section 25. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression us
Section 26. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern
Section 27. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed s
Section 28. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-
Section 29. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision En
Section 30. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 31. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, a
Section 32. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 33. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pay
Section 34. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapti
Section 35. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against a
Section 36. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual